

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №3

Контейнеризация приложения средствами Docker

Выполнил:

Ермаков Максим Олегович

Группа К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

В рамках лабораторной работы необходимо контейнеризировать написанное микросервисное приложение средствами Docker: реализовать Dockerfile для каждого сервиса, написать общий docker-compose.yml и настроить сетевое взаимодействие между контейнерами. Работа выполнена в папке labs/lab3.

Ход работы

Dockerfile для сервисов

Для каждого runtime-компонента создан отдельный Dockerfile. Все Dockerfile используют общий подход: устанавливают зависимости, собирают TypeScript-проект, копируют собранную папку dist и запускают конкретный JS-файл сервиса.

Компонент	Dockerfile
api-gateway	docker/Dockerfile.gateway
identity-service	docker/Dockerfile.identity
catalog-service	docker/Dockerfile.catalog
menu-service	docker/Dockerfile.menu
reservation-service	docker/Dockerfile.reservation
review-service	docker/Dockerfile.review

Общий Docker Compose

Общий docker-compose.yml поднимает инфраструктуру и все микросервисы одной командой. PostgreSQL и RabbitMQ имеют healthcheck, а прикладные сервисы стартуют после готовности необходимых инфраструктурных контейнеров.

Контейнер	Назначение	Порт хоста
db	PostgreSQL с отдельными БД сервисов	25432
rabbitmq	RabbitMQ и Management UI	5672, 15672
api-gateway	Публичный API	8200
identity-service	Пользователи, регистрация, вход и JWT	8201
catalog-service	Каталог ресторанов и consumer RabbitMQ	8202
menu-service	Категории и позиции меню	8203
reservation-service	Столики, доступность и бронирования	8204
review-service	Отзывы и producer RabbitMQ	8205

Сетевое взаимодействие

Все контейнеры находятся в одной Docker Compose network. Поэтому внутри контейнеров сервисы обращаются друг к другу не через 127.0.0.1, а через DNS-имена сервисов из docker-compose.yml. Например, gateway обращается к identity-service по адресу `http://identity-service:8201`, а review-service публикует события в RabbitMQ по адресу `amqp://maindb:maindb@rabbitmq:5672`.

Назначение	Адрес внутри Docker Compose
Identity API	<code>http://identity-service:8201</code>
Catalog API	<code>http://catalog-service:8202</code>
Menu API	<code>http://menu-service:8203</code>
Reservation API	<code>http://reservation-service:8204</code>
Review API	<code>http://review-service:8205</code>
PostgreSQL	<code>db:5432</code>
RabbitMQ	<code>rabbitmq:5672</code>

Внутри контейнеров сервисы слушают 0.0.0.0, чтобы Docker мог пробросить соответствующие порты наружу на хост. Для хоста публичная точка входа остается `http://127.0.0.1:8200/api/v1`.

Запуск и проверка

Вся система запускается одной командой `docker compose up --build -d`. После запуска можно проверить состояние контейнеров через `docker compose ps`, health endpoint gateway через `curl http://127.0.0.1:8200/health` и полный сценарий через `npm run smoke`.

Smoke-тест проходит создание ресторана, столика, меню, фото, публикацию ресторана, регистрацию пользователя, бронирование и создание отзыва. Дополнительно проверяется, что рейтинг ресторана обновился через RabbitMQ-событие из ДЗ5.

Вывод

В результате лабораторной работы микросервисное приложение контейнеризировано. Для каждого сервиса создан отдельный Dockerfile, общий docker-compose.yml поднимает PostgreSQL, RabbitMQ, api-gateway и все доменные сервисы, а сетевое взаимодействие настроено через имена сервисов внутри Docker Compose network.